

Higher Order Functions

CS-3180

Functional Programming

- Functional Paradigm is all about functions
- EVERYTHING is a function (as we've seen)
- First-class functions are pivotal to this
- We'll look at the limits of these implications

FP in non-functional languages

- Many languages support first-class functions
- Javascript supports many paradigm
- You don't need to know JS
- It will be helpful today in introducing some concepts

First Class Fns

- Functions are a type
- They can be saved as variables (without being called)
- Passed into functions (often called a "callback")

```
1 function greet() {  
2     console.log("Hello world!");  
3 }  
4  
5 // callbackFn is an argument (first class function)  
6 function repeatFn(callbackFn) {  
7     callbackFn();  
8     callbackFn();  
9 }  
10  
11 const fn = greet;  
12 repeatFn(fn);
```

First Class Fns

- Functions are a type
- They can be saved as variables (without being called)
- Passed into functions (often called a "callback")

```
1 function greet() {  
2     console.log("Hello world!");  
3 }  
4  
5 // callbackFn is an argument (first class function)  
6 function repeatFn(callbackFn) {  
7     callbackFn();  
8     callbackFn();  
9 }  
10  
11 const fn = greet;  
12 repeatFn(fn);
```

First Class Fns

- Functions are a type
- They can be saved as variables (without being called)
- Passed into functions (often called a "callback")

```
1 function greet() {  
2     console.log("Hello world!");  
3 }  
4  
5 // callbackFn is an argument (first class function)  
6 function repeatFn(callbackFn) {  
7     callbackFn();  
8     callbackFn();  
9 }  
10  
11 const fn = greet;  
12 repeatFn(fn);
```

First Class Fns

- Functions are a type
- They can be saved as variables (without being called)
- Passed into functions (often called a "callback")

```
1 function greet() {  
2     console.log("Hello world!");  
3 }  
4  
5 // callbackFn is an argument (first class function)  
6 function repeatFn(callbackFn) {  
7     callbackFn();  
8     callbackFn();  
9 }  
10  
11 const fn = greet;  
12 repeatFn(fn);
```

First Class Fns

- Functions are a type
- They can be saved as variables (without being called)
- Passed into functions (often called a "callback")

```
1 function greet() {  
2     console.log("Hello world!");  
3 }  
4  
5 // callbackFn is an argument (first class function)  
6 function repeatFn(callbackFn) {  
7     callbackFn();  
8     callbackFn();  
9 }  
10  
11 const fn = greet;  
12 repeatFn(fn);
```

"Lambda" Expressions

- Often, callbacks are implemented as lambdas
- (Javascript calls theses "arrow functions")

```
1 // callbackFn is an argument (first class function)
2 function repeatFn(callbackFn) {
3     callbackFn();
4     callbackFn();
5 }
6
7
8 // in-line lambda of function greet(){...}
9 const fn = () => console.log("Hello world!");
10
11 repeatFn(fn);
```

"Lambda" Expressions

- Often, callbacks are implemented as lambdas
- (Javascript calls theses "arrow functions")

```
1 // callbackFn is an argument (first class function)
2 function repeatFn(callbackFn) {
3     callbackFn();
4     callbackFn();
5 }
6
7
8 // in-line lambda of function greet(){...}
9 const fn = () => console.log("Hello world!");
10
11 repeatFn(fn);
```

"Lambda" Expressions

- Often, callbacks are implemented as lambdas
- (Javascript calls theses "arrow functions")

```
1 // callbackFn is an argument (first class function)
2 function repeatFn(callbackFn) {
3     callbackFn();
4     callbackFn();
5 }
6
7
8 // in-line lambda of function greet(){...}
9 const fn = () => console.log("Hello world!");
10
11 repeatFn(fn);
```

Inlining

- Equivalently, something like this
- Adding full {}s and indentation
- Consumer doesn't care (or even see) when parameter fn is called

```
1 // callbackFn is an argument (first class function)
2 function repeatFn(callbackFn) {
3     callbackFn();
4     callbackFn();
5 }
6
7 repeatFn(() => {
8     console.log("Hello world!")
9 });
```

Returning Functions

- Beyond accepting functions
- We can also `return` functions
- "Function Factories"*
- Useful in some circumstances

Returning Functions

- Returns a fn we can call later
- Pay attention to knowns vs. unknowns

```
1 function createMultiplier(factor) {  
2     // number is a parameter, not a value  
3     return function(number) {  
4         return number * factor;  
5     };  
6     // arrow fn equivalent  
7     // return (number) => number * factor  
8 }  
9  
10 const double = createMultiplier(2);  
11 const triple = createMultiplier(3);  
12  
13 console.log(double(5));  
14 console.log(triple(5));
```

Returning Functions

- Returns a fn we can call later
- Pay attention to knowns vs. unknowns

```
1 function createMultiplier(factor) {  
2     // number is a parameter, not a value  
3     return function(number) {  
4         return number * factor;  
5     };  
6     // arrow fn equivalent  
7     // return (number) => number * factor  
8 }  
9  
10 const double = createMultiplier(2);  
11 const triple = createMultiplier(3);  
12  
13 console.log(double(5));  
14 console.log(triple(5));
```

Returning Functions

- Returns a fn we can call later
- Pay attention to knowns vs. unknowns

```
1 function createMultiplier(factor) {  
2     // number is a parameter, not a value  
3     return function(number) {  
4         return number * factor;  
5     };  
6     // arrow fn equivalent  
7     // return (number) => number * factor  
8 }  
9  
10 const double = createMultiplier(2);  
11 const triple = createMultiplier(3);  
12  
13 console.log(double(5));  
14 console.log(triple(5));
```

Returning Functions

- Returns a fn we can call later
- Pay attention to knowns vs. unknowns

```
1 function createMultiplier(factor) {  
2     // number is a parameter, not a value  
3     return function(number) {  
4         return number * factor;  
5     };  
6     // arrow fn equivalent  
7     // return (number) => number * factor  
8 }  
9  
10 const double = createMultiplier(2);  
11 const triple = createMultiplier(3);  
12  
13 console.log(double(5));  
14 console.log(triple(5));
```

Returning Functions

- Returns a fn we can call later
- Pay attention to knowns vs. unknowns

```
1 function createMultiplier(factor) {  
2     // number is a parameter, not a value  
3     return function(number) {  
4         return number * factor;  
5     };  
6     // arrow fn equivalent  
7     // return (number) => number * factor  
8 }  
9  
10 const double = createMultiplier(2);  
11 const triple = createMultiplier(3);  
12  
13 console.log(double(5));  
14 console.log(triple(5));
```

Returning Functions

- Returns a fn we can call later
- Pay attention to knowns vs. unknowns

```
1 function createMultiplier(factor) {  
2     // number is a parameter, not a value  
3     return function(number) {  
4         return number * factor;  
5     };  
6     // arrow fn equivalent  
7     // return (number) => number * factor  
8 }  
9  
10 const double = createMultiplier(2);  
11 const triple = createMultiplier(3);  
12  
13 console.log(double(5));  
14 console.log(triple(5));
```

Higher-Order Functions

- Call a Higher-Order Function (HOF), a function that either:
- Accepts function as a parameter (callback)
- Returns a function

Higher-Order Functions

- Call a Higher-Order Function (HOF), a function that either:
- Accepts function as a parameter (callback)
- Returns a function
- Or both!

Higher-Order Functions

- Call a Higher-Order Function (HOF), a function that either:
- Accepts function as a parameter (callback)
- Returns a function
- Or both!
- This concept is how FP can scale for large programs

List Processing

- *Very* commonly used for processing lists
- Hence why these ideas mostly come from Lisp
- Let's start re-build the most common version of this (in JS)
- Work backwards into Lisp

Map Function

- Let's write a HOF that applies a function to every value in a list
- Traditional called "map"
- Have our function accept:
- The list to process
- Callback function to transform
- (We'll assume this function always "looks" like how we want)

Map Function

- Usually instead implemented as a method for List

```
1 function map(list, fn){
2     for (let i = 0; i < list.length; i++) {
3         // transform the ith elem
4         const newValue = fn(list[i])
5         list[i] = newValue
6     }
7     return list;
8 }
9
10 const list = [1, 2, 3, 4, 5]
11 const updatedList = map(list, (i) => {
12     return i + 1;
13 });
14
15 console.log(updatedList)
```

Map Function

- Usually instead implemented as a method for List

```
1 function map(list, fn){
2     for (let i = 0; i < list.length; i++) {
3         // transform the ith elem
4         const newValue = fn(list[i])
5         list[i] = newValue
6     }
7     return list;
8 }
9
10 const list = [1, 2, 3, 4, 5]
11 const updatedList = map(list, (i) => {
12     return i + 1;
13 });
14
15 console.log(updatedList)
```

Map Function

- Usually instead implemented as a method for List

```
1 function map(list, fn){
2     for (let i = 0; i < list.length; i++) {
3         // transform the ith elem
4         const newValue = fn(list[i])
5         list[i] = newValue
6     }
7     return list;
8 }
9
10 const list = [1, 2, 3, 4, 5]
11 const updatedList = map(list, (i) => {
12     return i + 1;
13 });
14
15 console.log(updatedList)
```

Map Function

- Usually instead implemented as a method for List

```
1 function map(list, fn){
2     for (let i = 0; i < list.length; i++) {
3         // transform the ith elem
4         const newValue = fn(list[i])
5         list[i] = newValue
6     }
7     return list;
8 }
9
10 const list = [1, 2, 3, 4, 5]
11 const updatedList = map(list, (i) => {
12     return i + 1;
13 });
14
15 console.log(updatedList)
```

Common HOFs

- Most languages have equivalents to these
- Common HOFs include:
 - `map(...)`
 - `filter(...)`
 - `reduce(...)`
 - `forEach(...)`
 - `every(...)`
 - `some(...)`

HOFs in Lisp

- HOFs come from Lisp
- Especially useful when recursively lists
- Let's see how this looks

Lisp HOFs

- Analyze this code
- You might *expect* this to work

```
1 (defun my-map (list fn)
2   (labels ((my-map-inner (remaining)
3             ; when lets us handle nil recursive case easier
4             (when remaining
5               (let ((new-value (fn (car remaining))))
6                 (cons
7                   new-value
8                   (my-map-inner (cdr remaining)))))))
9   (my-map-inner list)))
10
11 (defun add-one (value)
12   (+ value 1))
13
14 (format t "New List: ~S~%" (my-map '(1 2 3) add-one))
```

Lisp Namespacing

- The "Global Namespace" is really two namespaces
- One for functions, one for variables
- `(fn ...)` will lookup `fn` in function namespace
- `(funcall fn ...)` to fix this
- Also, we'll use `#'` when we pass `add-one` to get the function itself
- (Some dialects fix this behavior)

HOFs in Lisp

- Fixing that, you arrive at:

```
1 (defun my-map (list fn)
2   (labels ((my-map-inner (remaining)
3             ; when lets us handle nil recursive case easier
4             (when remaining
5               ; funcall -> look up in variable namespace
6               (let ((new-value (funcall fn (car remaining)))
7                 (cons
8                  new-value
9                  (my-map-inner (cdr remaining)))))))
10  (my-map-inner list)))
11
12 (defun add-one (value)
13   (+ value 1))
14
15 ; #' look up as function type
16 (format t "New List: ~S~%" (my-map '(1 2 3) #'add-one))
```

Standard Library HOF

- What we are really doing:
- Mapping the `car` of the list
- Through our function
- Common usecase, standard library has a function for this
- `mapcar`

Map Car

- Looks something like this:
- `mapcar` can also accept multiple lists
- Parallelizes the application

```
1
2 (defun add-one (value)
3   (+ value 1))
4
5 (defparameter *original-list* '(1 2 3))
6
7 (let ((new-list (mapcar #'add-one *original-list*)))
8   (format t "New list was: ~S~%" new-list))
9
```

Map Car

- We can use the `lambda` function if we don't want to `defun` globally

```
1
2 (defparameter *original-list* '(1 2 3))
3
4 (let
5   ((new-list
6     (mapcar #'(lambda (value) (+ 1 value)) *original-list*)))
7     (format t "New list was: ~S~%" new-list))
```

Other HOFs in Lisp

- `mapcar` will get 90% there in most cases
- A few others include:
- `maplist`: Like `mapcar`, but pass CDR each time
- `funcall`: Calls a function (which we already used)
- `apply`: Like `funcall`, but unpack arguments
- `reduce`: Fold elements into a value

OOP HOF implementations

- Such a useful construct, most OOP languages
- Attempt to implement something like this
- Take a brief look at how you might do something like this with objects

HOF OOP

- Standard Library Provides "Functional Interfaces"
- Just need to turn every HOF type into an interface

```
1 interface Mappable {
2     public int apply(int i);
3 }
4 // use generics instead of ints
5 public static List<Integer> map(List<Integer> list, Mappable
6     for(int i = 0; i < list.size(); i++){
7         int newValue = m.apply(
8             list.get(i)
9         );
10        list.set(newValue, i);
11    }
12    return list;
13 }
```

Streams

- In Java, you must convert Collection type into a "stream"
- Apply HOFs (interfaces impls.) to that stream
- Convert the stream back into a Collection
- (This is secretly a FP function called a Monad)
- Let's see it

```
1 List<String> names = Arrays.asList(
2     "John", "Sarah", "Jack", "Emily"
3 );
4 List<String> result = new ArrayList<>();
5
6 for (String name : names) {
7     if (name.startsWith("J")) {
8         result.add(name.toUpperCase());
9     }
10 }
11 System.out.println(result);
```

```
1 // transforms into this declarative version
2 List<String> names = Arrays.asList(
3     "John", "Sarah", "Jack", "Emily"
4 );
5
6 List<String> result = names.stream()
7     .filter(name -> name.startsWith("J"))
8     .map(String::toUpperCase) //wrap this method call in a lambda
9     .collect(Collectors.toList());
10
11 System.out.println(result);
```

Summary

- A function is higher-order if it:
- Accepts a function as a parameter
- Or returns a function (or both)
- Native to languages with first-class functions
- `mapcar` is the most common one in Lisp
- Not an idea exclusive to FP languages

Questions?